

Lektion 2

En Simpel Computer

Emil Lykke Diget

Computerarkitektur og Operativsystemer
Syddansk Universitet

- Lektion 1: Kombinatoriske Logiske Kredse
- Lektion 2: En Simpel Computer
- Lektion 3: Hukommelse
- Lektion 4: Mikroarkitektur
- Lektion 5: Micro-assembly og IJVM
- Lektion 6: Optimering af Mikroarkitektur

- Opskriv og reducer det boolske udtryk for denne tabel.
- Tegn et kombinatorisk logisk diagram.
- Hvilket komponent?

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Boolsk udtryk:

$$Y = \overline{A}B + A\overline{B}$$

Diagram på tavlen.

Repetition II

- Opskriv og reducer det boolske udtryk for denne tabel.
- Tegn et kombinatorisk logisk diagram.
- Hvilket komponent?

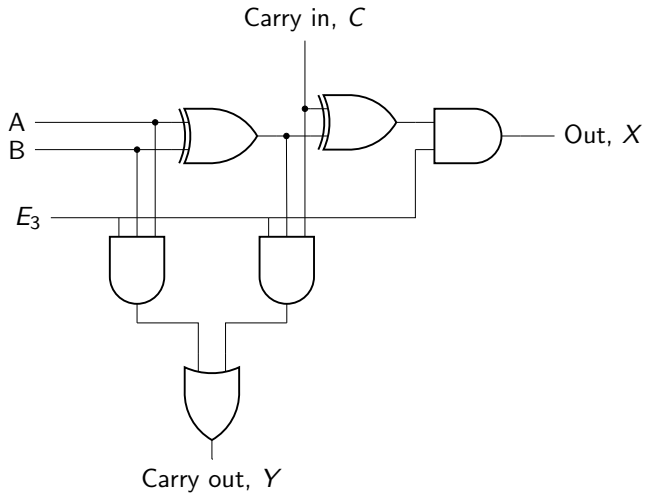
C	B	A	Y	X
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$\begin{aligned}X &= \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC \\&= \overline{C}(\overline{A}\overline{B} + \overline{A}B) + C(\overline{A}\overline{B} + AB) \\&= \overline{C}(\overline{A}\overline{B} + \overline{A}B) + C((A + B)(\overline{A} + \overline{B})) \\&= \overline{C}(\overline{A}\overline{B} + \overline{A}B) + C(\overline{A\overline{A}} + \overline{A\overline{B}} + \overline{B\overline{A}} + \overline{B\overline{B}}) \\&= \overline{C}(\overline{A}\overline{B} + \overline{A}B) + C(\overline{A\overline{B}} + \overline{B\overline{A}}) \\&= \overline{C}(A \oplus B) + C(\overline{A \oplus B}) \\&= C \oplus (A \oplus B)\end{aligned}$$

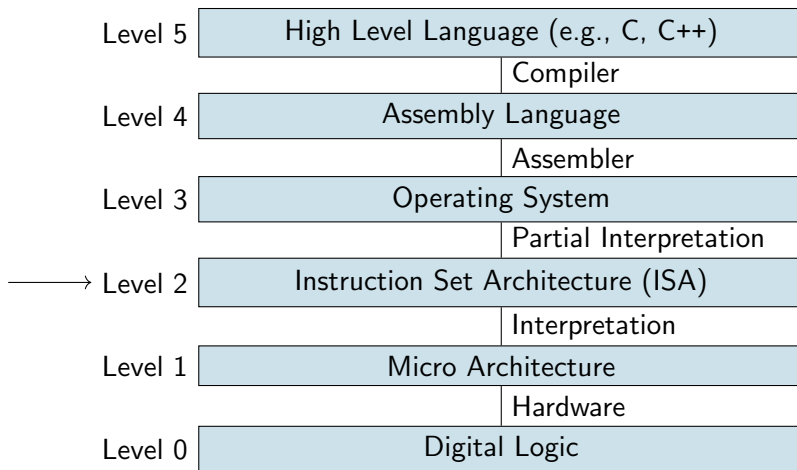
$$\begin{aligned}Y &= ABC\bar{C} + A\bar{B}C + \bar{A}BC + ABC \\&= C(A\bar{B} + \bar{A}B) + AB \\&= C(A \oplus B) + AB\end{aligned}$$

Repetition II

Løsning



Lagdelt Computermodel



Introduktion til Computerarkitektur

Von Neumann-arkitektur

Oversigt

En Simpel Computermodel

CPU

Instruktioner

Kontrolenhed

Design af Instruktionssæt

Opsummering

Referencer

Introduktion til Computerarkitektur

Von Neumann-arkitektur

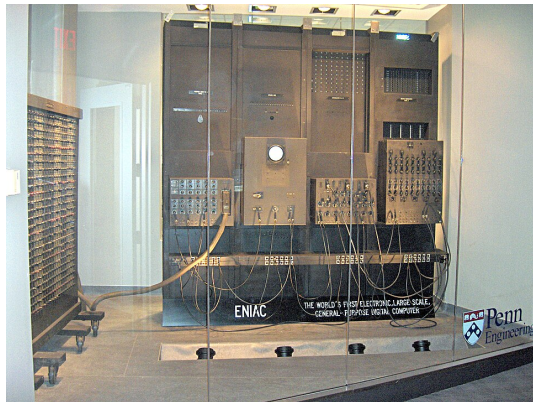
En Simpel Computermodel

Opsummering

Referencer

Tidligere var regnemaskiner bygget til at løse en **specifik** og **prædefineret** opgave.

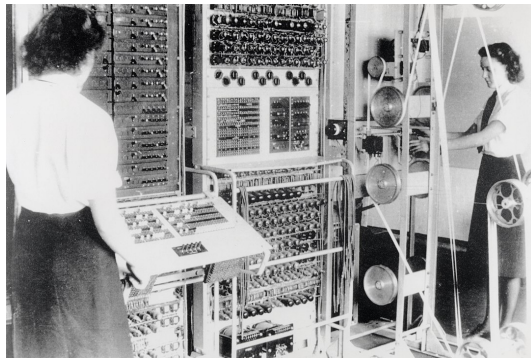
- Reprogrammering er en besværlig process, hvor det fysiske kredsløb skal flyttes rundt, kontakter skal de-/aktiveres og kabler skal kobles til og fra.



Figur: ENIAC (Electronic Numerical Integrator and Computer)

Tidligere var regnemaskiner bygget til at løse en **specifik** og **prædefineret** opgave.

- Reprogrammering er en besværlig process, hvor det fysiske kredsløb skal flyttes rundt, kontakter skal de-/aktiveres og kabler skal kobles til og fra.



Figur: Colossus

Introduktion til Computerarkitektur

Von Neumann-arkitektur

Oversigt

En Simpel Computermodel

Opsummering

Referencer

Von Neumann-arkitektur I

Omkring 1950 skrev John von Neumann en serie artikler, der introducerede von Neumann-arkitekturen.

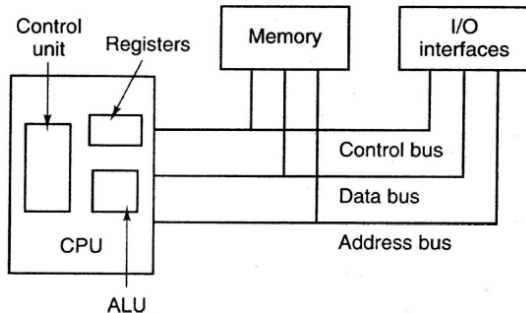
- [1] J. Von Neumann, „Theory and organization of complicated automata,“ **Burks (1966)**, s. 29–87, 1949
- [2] J. Von Neumann m.fl., „The general and logical theory of automata. 1951,“ **John von Neumann-Collected Works**, årg. 5, s. 288–326, 1951
- [3] J. Von Neumann, „The theory of automata: Construction, reproduction, homogeneity,“ **Burks (1966)**, s. 89–250, 1952

- Disse artikler var med til at forme strukturen af den moderne computer.
- I grunden handler det om, at programinstruktioner er gemt i hukommelsen ved siden af dataen.
- Program og data er altså i den samme hukommelse, og det er programmet, der beslutter, hvad der skal ske.
- Flexibilitet.
- Arkitekturen har ikke noget prædefineret formål.
- Reprogrammering.

Von Neumann-arkitektur: Oversigt

Bestående af tre uafhængige komponenter (delsystemer):

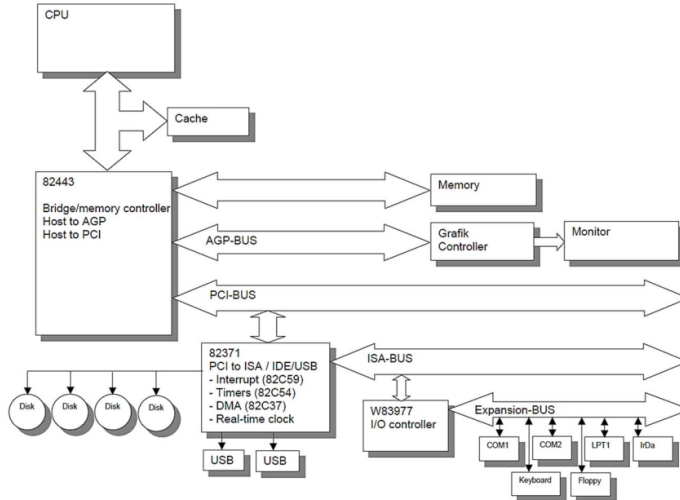
- Central processing unit (CPU)
 - ▶ “Hjernen”.
 - ▶ Kontrolenhed (control unit)
 - ▶ Registre
 - ▶ Arithmetic logic unit (ALU)
- Hukommelse (memory)
 - ▶ Gemmer program og data; random access memory (RAM) og read-only memory (ROM).
- Input/Output interface (I/O)
 - ▶ Del information med omverdenen; keyboard, skærm, printer, etc.



[4]

Von Neumann-arkitektur

Eksempel: Pentium II



Introduktion til Computerarkitektur

Von Neumann-arkitektur

En Simpel Computermodel

- CPU

- Instruktioner

- Kontrolenhed

- Design af Instruktionssæt

Opsummering

Referencer

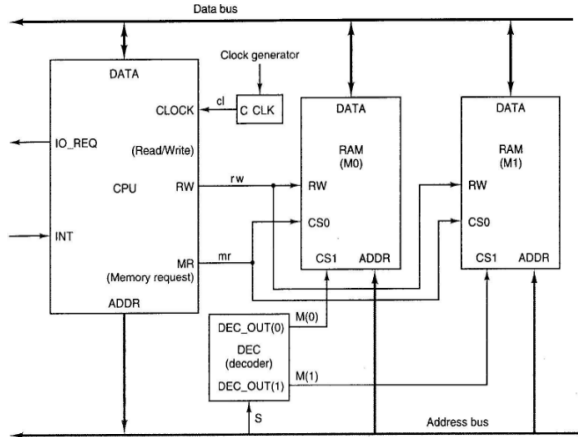
En Simpel Computermodel

Denne mikrocomputer består af:

- en CPU
- en clock generator
- en dekoder
- og to hukommelsesmoduler.

Hvert modul indeholder 8 adresser, hver 8-bit lange.

16 adresser i alt; adressebussen er 4-bit.



En Simpel Computermodel

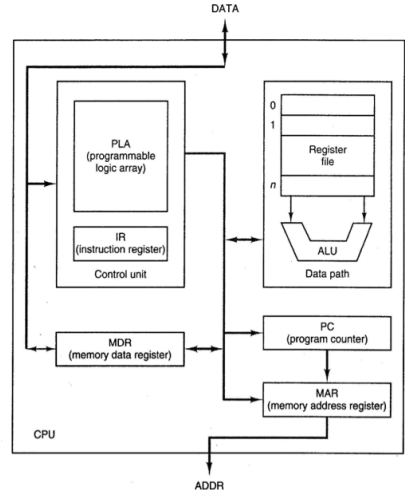
CPU

Både data og instruktioner er på data-bussen fra det samme hukommelse.

Data flyttes til *Memory Data Register* **MDR**.
Instruktioner der skal dekodes og eksekveres flyttes til *Instruction Register* **IR**.

Memory Address Register **MAR** udpeger adresser i hukommelsen, der skal læses eller skrives.

Program Counter **PC** holder styr på flowet ved at gemme adressen af den næste instruktion.



Generelt format af en instruktion:

Opcode	Operand	...	Operand
--------	---------	-----	---------

En Simpel Computermodel

Instruktioner

Vores microprocessor har kun fire instruktioner:

Instruktion	Opcode	Format	Betydning
-------------	--------	--------	-----------

En Simpel Computermode

Instruktioner

Vores microprocessor har kun fire instruktioner:

Instruktion	Opcode	Format						Betydning
		7	6	5	4	3	0	
LOAD	00	Opcode		R_d		Mem.adresse		$R_d \leftarrow \text{Mem.}(\text{adr.})$

En Simpel Computermode

Instruktioner

Vores microprocessor har kun fire instruktioner:

Instruktion	Opcode	Format						Betydning
		7	6	5	4	3	0	
LOAD	00	Opcode		R_d		Mem.adresse		$R_d \leftarrow \text{Mem.}(\text{adr.})$
		7	6	5	4	3	0	
STORE	01	Opcode		R_d		Mem.adresse		$\text{Mem.}(\text{adr.}) \leftarrow R_d$

En Simpel Computermode

Instruktioner

Vores microprocessor har kun fire instruktioner:

Instruktion	Opcode	Format				Betydning		
		7	6	5	4	3	0	
LOAD	00	Opcode		R_d		Mem.adresse		$R_d \leftarrow \text{Mem.}(\text{adr.})$
		7	6	5	4	3	0	
STORE	01	Opcode		R_d		Mem.adresse		$\text{Mem.}(\text{adr.}) \leftarrow R_d$
		7	6	5	4	3	0	
ADDR	10	Opcode		R_d		R_{s1}	R_{s2}	$R_d \leftarrow R_{s1} + R_{s2}$

En Simpel Computermode

Instruktioner

Vores microprocessor har kun fire instruktioner:

Instruktion	Opcode	Format	Betydning
LOAD	00	765430 OpcodeR_dMem.adresse	$R_d \leftarrow \text{Mem.}(\text{adr.})$
STORE	01	765430 OpcodeR_dMem.adresse	$\text{Mem.}(\text{adr.}) \leftarrow R_d$
ADDR	10	765430 Opcode$R_d$$R_{s1}$$R_{s2}$	$R_d \leftarrow R_{s1} + R_{s2}$
ADDM	11	765430 OpcodeR_dMem.adresse	$R_d \leftarrow R_d + \text{Mem.}(\text{adr.})$

En Simpel Computermode

Opgave (10 min.)

Skriv et program, der lægger to tal sammen, der ligger på adresse 13 og 14 og gem resultatet på adresse 15.

En Simpel Computermode

Opgave (10 min.)

Skriv et program, der lægger to tal sammen, der ligger på adresse 13 og 14 og gem resultatet på adresse 15.

LOAD	1, 13 ;	R1 <= mem(13)
ADDM	1, 14 ;	R1 <= R1 + mem(14)
STORE	1, 15 ;	mem(15) <= R1

En Simpel Computermodel

Program og data

Hukommelse

0	00 01 1101	} Program
1	11 01 1110	
2	01 01 1111	
⋮	⋮	
13	00000100	} Data
14	00000010	
15	????????	

```
LOAD  1, 13 ;    R1 <= mem(13)
ADDM   1, 14 ;    R1 <= R1 + mem(14)
STORE  1, 15 ;    mem(15) <= R1
```

Adresse og program er i hukommelsen samtidigt.

En Simpel Computermodel

Program og data

Hukommelse

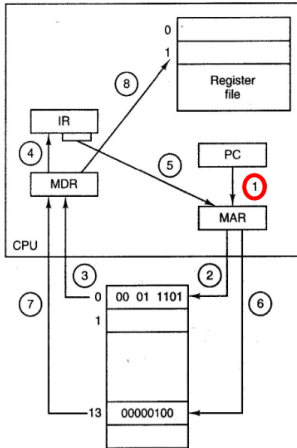
0	00 01 1101	} Program
1	11 01 1110	
2	01 01 1111	
⋮	⋮	
13	00000100	} Data
14	00000010	
15	00000110	

```
LOAD  1, 13 ;    R1 <= mem(13)
ADDM  1, 14 ;    R1 <= R1 + mem(14)
STORE 1, 15 ;    mem(15) <= R1
```

Adresse og program er i hukommelsen samtidigt.

En Simpel Computermodel

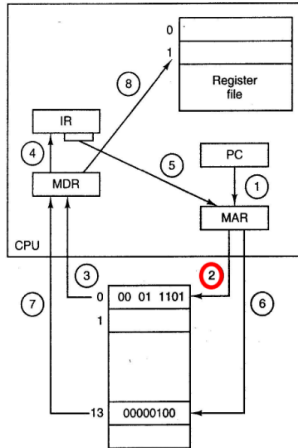
Eksekvering af LOAD



Indholdet af *Program Counter* ($PC = 0$) kopieres til *Memory Address Register* (MAR).

En Sempel Computermodel

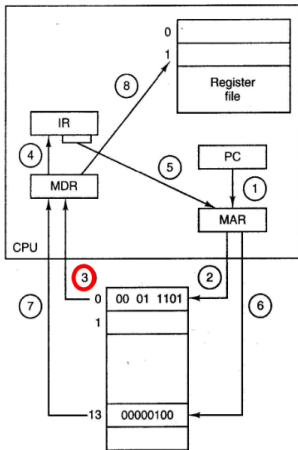
Eksekvering af LOAD



Memory Address Register (MAR) peger på den første instruktion i hukommelsen, her er det LOAD.

En Simpel Computermodel

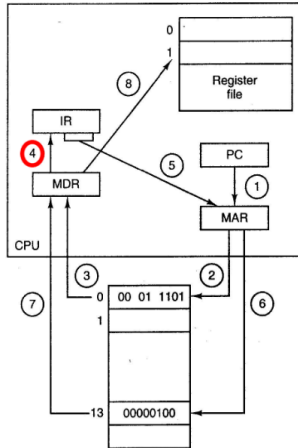
Eksekvering af LOAD



Kontrolenheden (Control Unit) anmoder om at læse indholdet på adresse 0 og bringer indholdet til *Memory Data Register* (MDR). Samtidigt inkrementeres værdien af PC.

En Simpel Computermodel

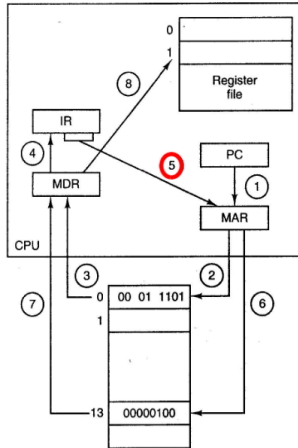
Eksekvering af LOAD



Indholdet af MDR kopieres til *Instruction Register* (IR) hvor instruktionen bliver dekoderet, altså 00 $\Rightarrow$ LOAD.

En Simpel Computermodel

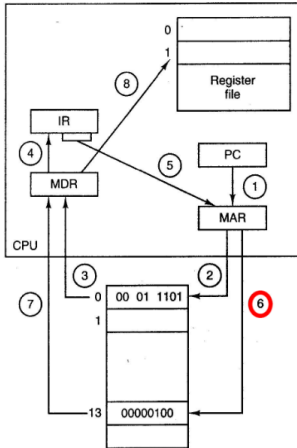
Eksekvering af LOAD



Kontrolenheden vil kopiere de 4 LSBs fra *instruktionsregisteret* (IR) til *Memory Address Register* (MAR) (1101 eller 13).

En Simpel Computermodel

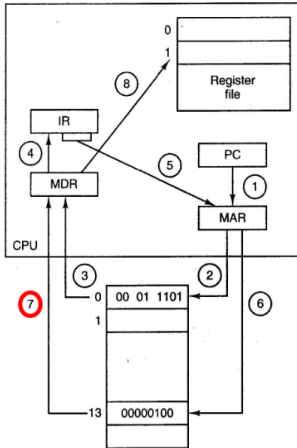
Eksekvering af LOAD



MAR peger nu på adresse 1101 (13).

En Simpel Computermodel

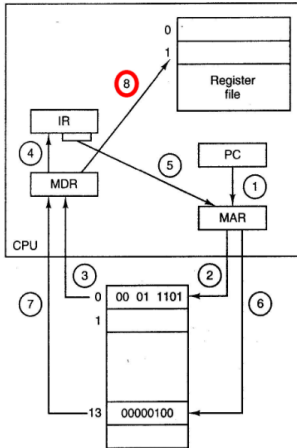
Eksekvering af LOAD



Kontrolenheden læser indholdet af adresse 13 og kopierer det til MDR.

En Simpel Computermodel

Eksekvering af LOAD

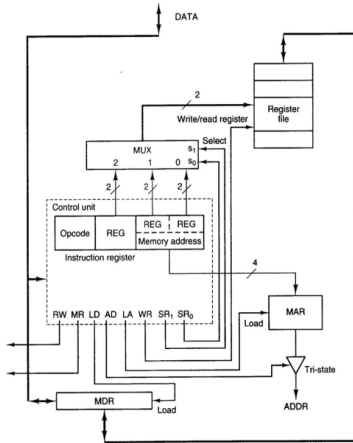


Bit 4 og 5 i IR bestemmer hvilket register, værdien skal gemmes i. Indholdet af MDR kopieres til register 1.

IR = 00 01 1101

En Simpel Computermode

Kontrolenheden (Control Unit)



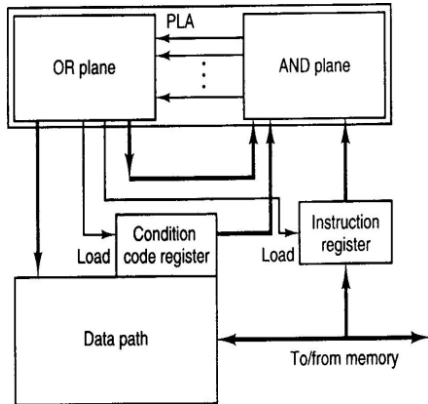
Kontrolenheden har forbindelser til registrene i CPUen.

Kontrolenheden eksekverer en sekvens af operationer, der korresponderer til en specific operation, f.eks. LOAD.

Denne sekvens kaldes for **mikro-instruktion**.

Kontrolenheden

Hardwired Circuit Design

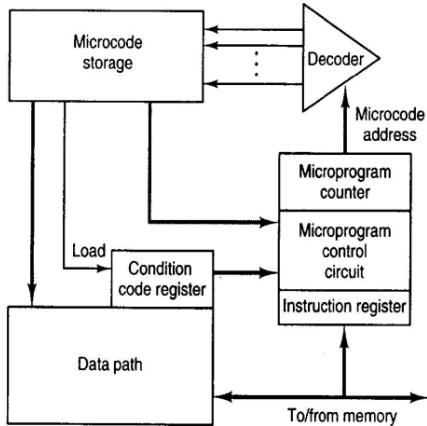


Her benyttes *Programmable Logic Array* (PLA) til at implementere kredsløbet.

- **Fordel:** Effektiv.
- **Ulempe:** Ufleksibel; ændringer i kontrolenheden kræver redesign.

Kontrolenheden

Mikroprogram-design



Hver instruktion eksekveres som en slags sub-rutine bestående af micro-instruktioner.

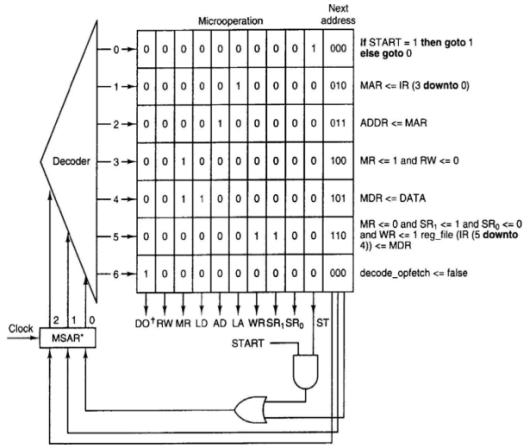
- **Fordel:** Flexibilitet.
En computer i en computer.
- **Ulempe:** Mindre effektiv.

Man skal tænke over følgende, når man designer størrelsen af mikroinstruktionerne (micro-instruction words):

- **Minimér** størrelsen af mikrokode.
- **Minimér** størrelsen af mikroprogram – længden af mikrokode-hukommelsen.
- **Maximér** fleksibiliteten til at tilføje og ændre mikroinstruktioner.
- **Maximér** parallel udførsel af mikroinstruktioner.

Horizontal Instruktionsdesign

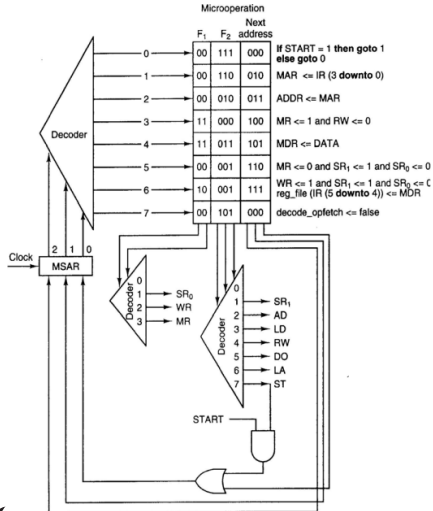
LOAD



- **Fordel:** Stor fleksibilitet. Kortere eksekveringstid.
- **Ulempe:** Bredere microword-størrelse.
- **Farlig:** Man kan aktivere flere kontrolsignaler på samme tid, der kan skade CPUen.

Vertikal Instruktionsdesign

LOAD



- **Fordel:** Smaller microword-størrelse.
- **Ulempe:** Mindre fleksibilitet. Længere eksekveringstid.
- Kontrolsignaler, der **ikke** må aktiveres samtidigt, kan grupperes i forskellige dekodere.

Design af instruktionssæt er vigtigt, når man designer en processor.

Opcode	Operand	...	Operand
--------	---------	-----	---------

Følgende skal overvejes:

- Hvor mange instruktioner er nødvendige?
- Hvilke slags operationer er nødvendige?
- Hvor mange operand-felter og hvilke typer skal tillades i hver instruktion?

Hvor mange instruktioner er nødvendige?

Kan hjælpe med at bestemme størrelsen af opcode-feltet.



- Flere instruktioner $\Leftrightarrow$ bredere opcode-felt.
- Hver operation fylder mere i hukommelsen.
- Et program kan skrives med færre og mere præcise instruktioner.

Leder til CISC-designet; Complex Instruction Set Computer.

Kompleksitet kan lede til at hver instruktion kan tage flere clock-cykler, men programmet i helhed kan være hurtigere at køre, da der er færre instruktioner.

Hvor mange instruktioner er nødvendige?

Kan hjælpe med at bestemme størrelsen af opcode-feltet.



- Færre instruktioner $\Leftrightarrow$ smallere opcode-felt.
- Hver operation fylder mindre i hukommelsen.
- Et program skal skrives med flere instruktioner for at have samme funktionalitet.

Leder til RISC-designet; Reduced Instruction Set Computer.

Simplicitet betyder at instruktioner ofte eksekveres i en clock-cyklus, så hver instruktion er hurtigere at eksekvere.

Hvor mange instruktioner er nødvendige?

Afvejning mellem at implementere funktionalitet som. . .

Instruktion i CPU	Subroutine i program
Leder til CISC	Leder til RISC
Kort access-tid	Længere access-tid
Langsommere end RISC pga. kompleksitet	Hurtigere end CISC pga. simplicitet
Mindre program	Større program (med subroutiner eller biblioteker)
Større strømforbrug	Mindre strømforbrug

Hvilke slags operationer er nødvendige?

Instruktionssættet kan variere mellem forskellige processorer. De fleste vil have operationer, der kan kategoriseres som følger:

- **Datatransfer**: Flyt (el. kopier) data fra en lokation til en anden.
- **Aritmetik**: Simple aritmiske operationer.
- **Logik**: Booleske operationer.
- **Kontrol**: Relateret til instruktionseksekvering, f.eks. SKIP, RETURN, HALT.
- **System**: Operativsystem-instruktioner, f.eks. systemkald.
- **I/O**: Flyt data ml. hukommelse og eksterne enheder.

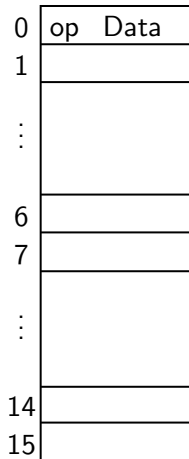
Hvor mange operand-felter og hvilke typer skal tillades i hver instruktion?

Størrelsen af et operand-felt er typisk meget begrænset.

Dog er det nødvendigt at referere til en stor mængde hukommelse.

Dette kan løses ved at bruge forskellige adresse-modes. De mest normale (på engelsk):

Adresserings-type	Beskrivelse
Immediate	Data
Direct	Adresse til data
Indirect	Adresse til adresse til data
Displacement (indexed)	Adresse til offset og lokal adresse
Stack	En slags indirect-type



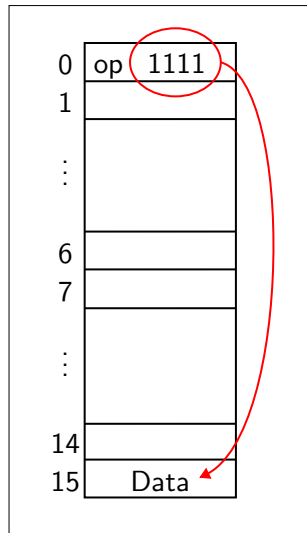
Hvor mange operand-felter og hvilke typer skal tillades i hver instruktion?

Størrelsen af et operand-felt er typisk meget begrænset.

Dog er det nødvendigt at referere til en stor mængde hukommelse.

Dette kan løses ved at bruge forskellige adresse-modes. De mest normale (på engelsk):

Adresserings-type	Beskrivelse
Immediate	Data
Direct	Adresse til data
Indirect	Adresse til adresse til data
Displacement (indexed)	Adresse til offset og lokal adresse
Stack	En slags indirect-type

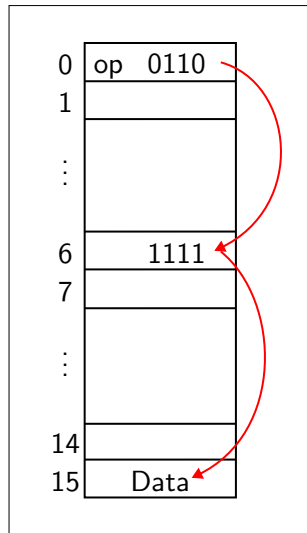


Hvor mange operand-felter og hvilke typer skal tillades i hver instruktion?

Størrelsen af et operand-felt er typisk meget begrænset. Dog er det nødvendigt at referere til en stor mængde hukommelse.

Dette kan løses ved at bruge forskellige adresse-modes. De mest normale (på engelsk):

Adresserings-type	Beskrivelse
Immediate	Data
Direct	Adresse til data
Indirect	Adresse til adresse til data
Displacement (indexed)	Adresse til offset og lokal adresse
Stack	En slags indirect-type

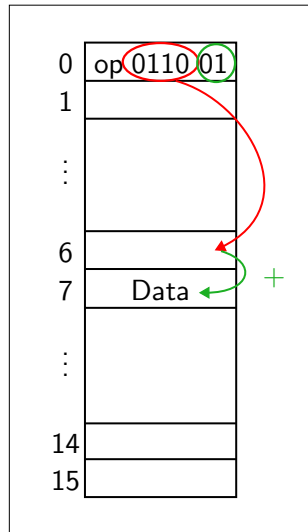


Hvor mange operand-felter og hvilke typer skal tillades i hver instruktion?

Størrelsen af et operand-felt er typisk meget begrænset. Dog er det nødvendigt at referere til en stor mængde hukommelse.

Dette kan løses ved at bruge forskellige adresse-modes. De mest normale (på engelsk):

Adresserings-type	Beskrivelse
Immediate	Data
Direct	Adresse til data
Indirect	Adresse til adresse til data
Displacement (indexed)	Adresse til offset og lokal adresse
Stack	En slags indirect-type



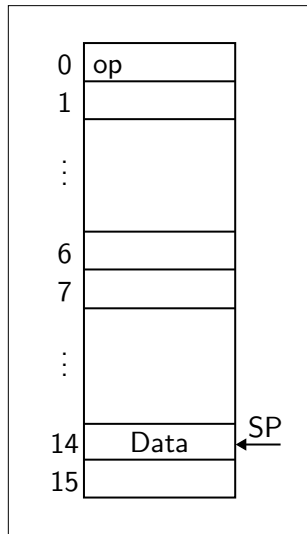
Hvor mange operand-felter og hvilke typer skal tillades i hver instruktion?

Størrelsen af et operand-felt er typisk meget begrænset.

Dog er det nødvendigt at referere til en stor mængde hukommelse.

Dette kan løses ved at bruge forskellige adresse-modes. De mest normale (på engelsk):

Adresserings-type	Beskrivelse
Immediate	Data
Direct	Adresse til data
Indirect	Adresse til adresse til data
Displacement (indexed)	Adresse til offset og lokal adresse
Stack	En slags indirect-type



Introduktion til Computerarkitektur

Von Neumann-arkitektur

En Simpel Computermode

Opsummering

Referencer

- Von-Neumann-arkitektur; en general (simpel) computer bestående af CPU, hukommelse og I/O.
- En simpel CPU
- Funktionen af kontrolenheden og dens design af mikrokode.
- Design af instruktionssæt.
- Adressering af hukommelse.

- [1] J. Von Neumann, „Theory and organization of complicated automata,“ **Burks (1966)**, s. 29–87, 1949.
- [2] J. Von Neumann m.fl., „The general and logical theory of automata. 1951,“ **John von Neumann-Collected Works**, årg. 5, s. 288–326, 1951.
- [3] J. Von Neumann, „The theory of automata: Construction, reproduction, homogeneity,“ **Burks (1966)**, s. 89–250, 1952.
- [4] M. R. Zargham, **Computer architecture: single and parallel systems**. Prentice-Hall, Inc., 1996.